

Cognitive Alarm System

Backend & Authentication

Milestone 1: User Registration and Login with JWT Authentication

Shrutipriya Hosmani

Table of Contents

1. Project Description
2. Scope of Milestone 1
3. Environment Setup
4. System Design
5. Database Schema
6. Implementation Details
7. API Endpoints
8. Testing & Verification
9. Conclusion

1. Project Description

“Nuera – Wake your mind” is a full-stack application designed to help users build better waking habits through scheduled alarms paired with cognitive verification challenges. The system is divided across four functional modules, developed in parallel by separate team members.

This document covers the work completed under my responsibility: Backend and Authentication. The backend module provides the foundational user management layer that all other modules (Alarms, AI/Analytics, and Frontend) depend on for identifying and authorizing users.

2. Scope of Milestone 1

Milestone 1 focuses on the smallest complete, testable slice of the authentication system: user registration and login, secured with JSON Web Tokens (JWT). Protected routes, role-based access control, and profile management are intentionally deferred to Milestone 2, so that the core authentication flow can be built, tested, and verified independently before further complexity is added.

- **In scope:** User registration, password hashing, user login, and JWT token generation.
- **Out of scope (Milestone 2):** Protected routes, token verification middleware, role-based access (admin/user), and profile update APIs.

3. Environment Setup

The backend service was developed locally using the following environment and tools:

- **Language:** Python 3.11 (Windows)
- **Framework:** FastAPI — chosen for its speed, automatic interactive documentation (Swagger UI), and native Pydantic-based data validation.
- **Database:** PostgreSQL 16, managed locally via pgAdmin 4.
- **ORM:** SQLAlchemy — used to define the User table and manage database sessions without writing raw SQL.
- **Authentication libraries:** python-jose for JWT encoding/decoding, and passlib with bcrypt for secure password hashing.
- **IDE:** Visual Studio Code, with Git Bash as the integrated terminal.

- **Version control:** Git and GitHub, following a feature-branch workflow (one branch per team member).

Key dependencies, pinned in requirements.txt for reproducibility:

```
fastapi==0.111.0
uvicorn[standard]==0.30.1
sqlalchemy==2.0.30
psycopg2-binary==2.9.9
python-jose[cryptography]==3.3.0
passlib[bcrypt]==1.7.4
bcrypt==4.0.1
python-multipart==0.0.9
pydantic-settings==2.3.4
```

4. System Design

The authentication flow follows a standard stateless JWT pattern. Rather than storing session data server-side, the server issues a signed token on successful login, which the client stores and presents on subsequent requests. This design was chosen for scalability and simplicity, and because it integrates cleanly with the other modules (Alarm, AI/Analytics, Frontend) being built in parallel by teammates.

4.1 Authentication Flow

- User submits registration details (username, email, password) via POST /auth/register.
- Server validates the input, checks for existing users with the same email/username, hashes the password using bcrypt, and stores the new user record.
- User submits credentials via POST /auth/login.
- Server verifies the password against the stored hash, and if valid, issues a signed JWT access token containing the user's ID and role.
- The token is returned to the client, to be used as a Bearer token on protected requests in later milestones.

4.2 Project Structure

```
app/
|-- main.py           FastAPI application endpoint
|-- config.py         Loads environment variables (.env)
|-- database.py        SQLAlchemy engine and session setup
|-- models.py         User table definition
|-- schemas.py        Pydantic request/response models
|-- auth.py           Password hashing and JWT creation
|-- routes/
    |-- auth_routes.py /auth/register, /auth/login
```

5. Database Schema

A single users table was defined to support Milestone 1, with fields reserved for role-based access to be used in Milestone 2.

Column	Type	Description
id	Integer (PK)	Auto-incrementing primary key
username	String(50)	Unique username
email	String(120)	Unique email address, used for login
hashed_password	String(255)	Bcrypt-hashed password (never stored in plain text)
role	Enum (user/admin)	Defaults to 'user'; used for access control in Milestone 2
created_at	DateTime	Timestamp set automatically on record creation

6. Implementation Details

6.1 Password Security

Passwords are never stored or transmitted in plain text. On registration, the plain-text password is passed through bcrypt's hashing algorithm via passlib before being persisted to the database. On login, the submitted password is verified against the stored hash using a constant-time comparison, preventing timing attacks.

6.2 JWT Token Generation

On successful login, a JWT is generated containing the user's ID and role as payload claims, along with a fixed expiry (default 60 minutes). The token is signed using a symmetric secret key (HS256 algorithm), which is loaded from an environment variable and never committed to source control.

6.3 Configuration Management

All sensitive configuration (database credentials, secret key) is loaded via a .env file, kept out of version control through .gitignore. A .env.example template is committed instead, so teammates know which variables are required without exposing real credentials.

7. API Endpoints

Method	Endpoint	Description	Response
POST	/auth/register	Creates a new user with a hashed password	201 Created
POST	/auth/login	Verifies credentials and returns a JWT	200 OK

Sample request body for registration:

```
{
  "username": "testuser",
  "email": "test@example.com",
  "password": "TestPass123"
}
```

Sample response for login:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

8. Testing & Verification

All endpoints were tested manually using FastAPI's auto-generated Swagger UI (available at /docs) against a local PostgreSQL instance. The following cases were verified:

- Registration with valid, unique details returns 201 Created and the new user record.
- Registration with a duplicate email or username is rejected with 400 Bad Request.
- Login with correct credentials returns 200 OK and a valid JWT access token.
- Login with an incorrect password returns 401 Unauthorized, without exposing whether the email or password was the invalid field.
- Database inspection via pgAdmin confirmed that stored passwords are bcrypt hashes, not plain text, for every test record created.

9. Conclusion

Milestone 1 successfully delivers a working, tested authentication foundation for the Cognitive Alarm System: user registration with securely hashed passwords, and login with JWT-based token issuance. All endpoints were verified end-to-end through Swagger UI and direct database inspection, confirming correct behavior for both valid and invalid inputs.

This foundation unblocks the next phase of backend work — protected routes, role-based access control, and user profile APIs — which will be addressed in Milestone 2, alongside integration with the Alarm, AI/Analytics, and Frontend modules being developed in parallel by the rest of the team.